

NEXAFORGE AI

Compound AI Evaluation Methodology

A layered evaluation system for models, retrieval, tools, agents, safety, and business outcomes

Document ID	Version	Owner	Classification
NF-AI-007	2.0	AI Evaluation and Reliability	Confidential - Internal Use

Purpose

Define how NexaForge AI evaluates compound systems composed of models, retrieval, prompts, tools, memory, and orchestration. The methodology prevents teams from hiding component failures behind a single end-to-end score and supports release decisions based on quality, risk, latency, and cost.

Keywords: evaluation, LLM evals, agents, RAG, safety, regression, scorecards

Document Control

Field	Value
Status	Approved methodology baseline
Effective date	21 July 2026
Review cycle	Every 12 months or upon material change
Primary owner	AI Evaluation and Reliability
Related standards	Security, privacy, software delivery, incident response, and data governance standards
Supersedes	Version 1.0 concise edition

How to Use This Document

Teams should use this document as a design and review guide, not as a substitute for engineering judgment. Sections describe mandatory controls, recommended practices, decision points, and evidence expected at release. Tailoring is permitted when documented and approved by the accountable owner. Any deviation that increases risk must include compensating controls and a review date.

1. Purpose and Objectives

Define how NexaForge AI evaluates compound systems composed of models, retrieval, prompts, tools, memory, and orchestration. The methodology prevents teams from hiding component failures behind a single end-to-end score and supports release decisions based on quality, risk, latency, and cost.

2. Scope and Applicability

This standard applies to new implementations, major changes, vendor replacements, and material expansions of systems that rely on compound ai evaluation methodology. It covers prototypes that process real organizational data, pre-production pilots, and production services. Small experiments using synthetic data may follow a reduced process, but they must still document ownership, purpose, and data handling. The methodology does not replace legal, security, privacy, or domain-specific requirements; instead, it provides the engineering structure through which those requirements are implemented and verified.

3. Core Principles

Evaluate at multiple layers: Component, interaction, workflow, and business outcomes each require distinct tests.

Use severity-weighted decisions: A rare catastrophic error can outweigh many minor improvements.

Combine automated and human judgment: Neither alone captures correctness, usefulness, and harm.

Datasets must evolve: Production failures and new use cases become regression cases.

Reproducibility matters: Record model versions, prompts, tools, data snapshots, and evaluator settings.

4. Lifecycle and Detailed Procedure

1. Capability decomposition

Map the system into retrieval, classification, planning, generation, tool use, validation, memory, and user interaction capabilities.

- Required evidence: a reviewable artifact or trace showing that capability decomposition was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

2. Evaluation objectives

Define success criteria and unacceptable failures for each capability and for the complete workflow.

- Required evidence: a reviewable artifact or trace showing that evaluation objectives was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

3. Dataset construction

Build representative, boundary, adversarial, multilingual, and longitudinal cases. Separate development, validation, and hidden release sets.

- Required evidence: a reviewable artifact or trace showing that dataset construction was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

4. Gold and rubric design

Create reference answers where feasible and structured rubrics where multiple outputs can be valid. Define severity and reviewer guidance.

- Required evidence: a reviewable artifact or trace showing that gold and rubric design was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Automated evaluation

Run exact checks, schema validation, retrieval metrics, citation entailment, task simulators, deterministic calculations, and calibrated model judges.

- Required evidence: a reviewable artifact or trace showing that automated evaluation was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

6. Human evaluation

Use trained reviewers for usefulness, nuance, safety, and cases where automated judgments are uncertain. Measure inter-rater agreement.

- Required evidence: a reviewable artifact or trace showing that human evaluation was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

7. System simulation

Evaluate multi-step tasks with controlled tools and state. Inject timeouts, malformed outputs, missing documents, and conflicting evidence.

- Required evidence: a reviewable artifact or trace showing that system simulation was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

8. Statistical comparison

Use paired comparisons, confidence intervals, bootstrap estimates, and segment analysis. Avoid releasing based on small noisy improvements.

- Required evidence: a reviewable artifact or trace showing that statistical comparison was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

9. Release gating

Apply minimum thresholds, no-regression constraints, severe-failure limits, latency and cost budgets, and mandatory review for changed risk profiles.

- Required evidence: a reviewable artifact or trace showing that release gating was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

10. Production feedback

Sample traces, analyze user corrections, cluster failures, and convert validated incidents into test cases.

- Required evidence: a reviewable artifact or trace showing that production feedback was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Entry Criteria

Work may enter the methodology when a named owner, defined user outcome, initial data classification, and target environment are available. Teams should also identify downstream consumers, irreversible actions, expected traffic, and the cost of wrong answers. If these inputs are unknown, the first deliverable is a discovery brief rather than an implementation.

6. Design Review Expectations

The design review should focus on concrete decisions and evidence. Reviewers inspect architecture diagrams, state or data schemas, model and tool dependencies, evaluation plans, privacy boundaries, fallback behavior, and operating metrics. Open issues must have owners and dates. A design is not approved merely because a model can demonstrate the happy path; the team must show how the system behaves with missing inputs, conflicting evidence, unavailable services, malformed outputs, and unauthorized requests.

7. Documentation and Traceability

Each release maintains a versioned record of requirements, decisions, prompts or policies, model identifiers, data or index versions, test results, known limitations, and approvers. Traceability should allow an incident responder to determine which configuration produced a specific output. Documentation must be concise enough to maintain but detailed enough for a new engineer to reproduce critical behavior.

8. Change Management

Changes are categorized as low, medium, or high impact. Low-impact changes include wording or monitoring adjustments with no behavior change. Medium-impact changes affect routing, prompts, thresholds, or non-sensitive data. High-impact changes include model replacement, new write capabilities, new sensitive data, changed jurisdictions, or altered decision authority. Medium and high-impact changes require targeted regression testing; high-impact changes repeat the relevant risk and approval reviews.

9. Operational Readiness

Before launch, the team confirms monitoring, alert ownership, on-call procedures, rollback, rate limits, access controls, data retention, incident communication, and user support. Synthetic probes should continuously test critical paths. Runbooks should describe both technical recovery and the user-facing behavior during degradation.

10. Continuous Improvement

Production evidence is reviewed on a regular cadence. Teams analyze failures by root cause rather than by surface symptom, distinguish model errors from retrieval, tool, data, policy, or interface errors, and prioritize improvements by user harm and frequency. Confirmed defects become regression tests. Metrics are segmented by use case, language, tenant, and risk tier to avoid hiding poor performance inside global averages.

12. Roles and Responsibilities

Role	Accountability
Evaluation lead	Owns methodology and release scorecards.
Domain annotator	Creates gold labels and reviews outputs.
System engineer	Provides reproducible builds and traces.
Safety reviewer	Owns harmful and adversarial test suites.
Product analyst	Connects technical metrics to user and business outcomes.

13. Measurement Framework

Metric	Definition and Use
Task success	End-to-end completion against explicit criteria.
Component accuracy	Correctness of classifiers, retrievers, planners, and validators.
Severe failure rate	Weighted frequency of high-impact failures.
Human preference	Reviewer preference under blinded comparison.
Calibration	Relationship between confidence and observed correctness.
Robustness delta	Performance degradation under perturbations.
Cost-quality frontier	Quality achieved at different latency and cost levels.
Regression coverage	Known defect classes represented in the suite.

Metrics must be segmented by use case and risk tier. Teams should define a baseline, target, alert threshold, and owner for each production metric. A metric without an action rule is considered observational rather than operational.

14. Worked Example

A research assistant improves answer style after a prompt change. End-to-end preference rises by four points, but citation entailment falls and two adversarial cases produce unsupported medical claims. Because severe failures and citation regression violate release gates, the change is rejected. Engineers isolate the style improvement and reimplement it without weakening grounding instructions.

15. Risk Register and Controls

Risk	Primary Controls
Judge bias	Calibrate model judges against human labels and rotate evaluator models.
Test leakage	Keep hidden sets and refresh examples.
Metric gaming	Use multiple independent measures and inspect traces.
Unrepresentative data	Sample real queries and maintain segment coverage.
Reviewer inconsistency	Train reviewers, use rubrics, and measure agreement.
Silent system changes	Version all dependencies and rerun gates after model, index, or tool updates.

16. Release Gate

- A named business and technical owner has approved the intended use and operating boundaries.
- Required architecture, security, privacy, and risk reviews are complete.
- Evaluation results meet minimum thresholds and no severe unresolved defect remains.
- Monitoring, alerting, rollback, and incident procedures have been tested.
- User-facing disclosures, approval checkpoints, and fallback behavior are implemented where required.
- All model, prompt, tool, data, index, and policy versions are recorded in the release manifest.

17. Review Checklist

- Is the user outcome specific, measurable, and appropriate for AI assistance?
- Are authority boundaries and prohibited actions explicit?
- Can every important output be traced to inputs, evidence, and configuration?
- Does the design handle missing, conflicting, stale, or malicious inputs?
- Are sensitive data, permissions, retention, and deletion controlled?
- Are severe failures represented in the evaluation suite?
- Can the service degrade or stop safely when dependencies fail?
- Are metrics connected to owners and response procedures?

18. Appendices

Evaluation plan template

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Compound AI Evaluation Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Severity rubric

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Compound AI Evaluation Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Human review guide

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Compound AI Evaluation Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Release scorecard example

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Compound AI Evaluation Methodology in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence

- Known limitations and follow-up actions

Revision History

Version	Date	Summary
1.0	21 July 2026	Initial concise methodology edition.
2.0	21 July 2026	Expanded edition with detailed lifecycle, controls, roles, metrics, examples, risks, release gates, and appendices.